

SIMATS ENGINEERING

ASSESSMENT TOOL 2 - Medium (LABORATORY PERFORMANCE)

COURSE CODE: CSA09

**COURSE NAME: Programming in
Java**

COURSE OUTCOME COVERED

CO4: *Design and develop GUI-based user interfaces using Abstract Window Toolkit, Swing, and Applets by implementing event handling, layout managers, AWT controls, and menus. (BL3,BL5)*

Assessment Tool Weightage: 10%

QUESTIONS: 1

Total Marks: 50

ANNEXURE A

LABORATORY PERFORMANCE

Code of Conduct:

I Rashmi Naurene (192521357) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Rashmi

ANNEXURE

LABORATORY PERFORMANCE

1. Student Registration Applet

Develop a Java Applet for a university student registration form. The applet should display labels and input controls for Student Name, Register Number, Department, Gender, and Email ID. The application should use appropriate **AWT controls** such as Label, TextField, Checkbox, CheckboxGroup, and Button. Use a suitable **layout manager** to organize the controls. When the student clicks the **Submit** button, the entered information should be displayed within the applet window.

The student must demonstrate:

1. Applet initialization and architecture.
2. AWT controls.
3. Layout management.
4. Button event handling.
5. Appropriate event listener interface.
6. Display of user-entered information.

2. Simple Calculator Applet

Develop an Applet-based calculator that accepts two numbers and provides buttons for **Addition, Subtraction, Multiplication, and Division**.

The application should use TextField, Label, and Button controls. Appropriate event listeners must be implemented so that clicking each button performs the corresponding operation.

The program must handle invalid numeric input and division by zero appropriately.

The student must demonstrate:

1. Basic Applet structure.
2. AWT controls.
3. Event sources and event objects.
4. ActionListener.
5. Exception handling.
6. Display methods.

3. Course Selection Applet

Develop a university course selection Applet where students can select:

- Department
- Semester
- Course
- Elective type

Use Choice, List, Checkbox, and Button controls wherever appropriate.

When the student clicks **Register**, display the selected courses and other information.

Use appropriate layout managers and event listeners.

4. Traffic Signal Control Applet

Develop an AWT Applet that simulates a traffic signal with three options:

- Red
- Yellow
- Green

Use appropriate controls to allow the user to select a signal. When the user selects a signal, display the corresponding message such as **STOP**, **WAIT**, or **GO**.

Use appropriate event sources, event classes, and listener interfaces.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Program Design	20%	Clearly understands the problem and develops an effective design	Good understanding and logical design	Basic understanding with some gaps	Poor understanding and unclear design
Java Implementation & Concept Application	30%	Applies Java concepts correctly and effectively	Applies concepts with minor errors	Limited application of concepts	Incorrect or incomplete implementation
Program Execution & Output	20%	Program executes correctly and produces accurate output	Executes with minor errors	Partially executes with some errors	Fails to execute correctly
Testing & Debugging	15%	Thoroughly tests and resolves errors independently	Tests and resolves most errors	Performs limited testing and debugging	Unable to test or debug effectively

Documentation & Presentation	15%	Complete, clear, and well-organize d documentati on	Good documentation with minor gaps	Basic documentati on with some gaps	Incomplete or unclear documentation
---	------------	--	--	--	---

Q1 Student Registration Applet

40 marks

Develop a Java Applet for a university student registration form. The applet should display labels and input controls for Student Name, Register Number, Department, Gender, and Email ID. The application should use appropriate AWT controls such as Label, TextField, Checkbox, CheckboxGroup, and Button. Use a suitable layout manager to organize the controls. When the student clicks the Submit button, the entered information should be displayed within the applet window.

The student must demonstrate:

1. Applet initialization and architecture.
2. AWT controls.
3. Layout management.
4. Button event handling.
5. Appropriate event listener interface.
6. Display of user-entered information.

UNIVERSITY REGISTRATION

Student Name:

rashmi

Register Number:

192521957

Department:

IT

Gender:

☐ Male ☒ Female

Email ID:

jalveyys@gmail.com

Submit Registration

REGISTRATION COMPLETED SUCCESSFULLY

Name: rashmi

Register No: 192521357

Department: IT

Gender: Female

Email ID: jahveyvs@gmail.com

```
import java.applet.Applet;
```

```
import java.awt.*;
```

```
import java.awt.event.ActionEvent;
```

```
import java.awt.event.ActionListener;
```

```
/*
```

```
<applet code="StudentRegistrationApplet.class" width="520" height="580">
```

</applet>

*/

```
public class StudentRegistrationApplet extends Applet implements ActionListener {
```

```
    // UI Controls
```

```
    Label lblTitle, lblName, lblRegNo, lblDept, lblGender, lblEmail;
```

```
    TextField txtName, txtRegNo, txtDept, txtEmail;
```

```
    CheckboxGroup genderGroup;
```

```
    Checkbox chkMale, chkFemale;
```

```
    Button btnSubmit;
```

```
    // Form tracking variables
```

```
    String displayName = "";
```

```
    String displayRegNo = "";
```

```
    String displayDept = "";
```

```
    String displayGender = "";
```

```
    String displayEmail = "";
```

```
    boolean isSubmitted = false;
```

```
    @Override
```

```
    public void init() {
```

```
        // Use GridBagLayout with ample padding to balance elements beautifully
```

```
        setLayout(new GridBagLayout());
```

```
        GridBagConstraints gbc = new GridBagConstraints();
```

```
        gbc.insets = new Insets(10, 15, 10, 15);
```

```
        gbc.fill = GridBagConstraints.HORIZONTAL;
```

```
        // Custom Typography Styles
```

```
        Font labelFont = new Font("Segoe UI", Font.BOLD, 13);
```

```
        Font inputFont = new Font("Segoe UI", Font.PLAIN, 13);
```

```
// Invisible title spacing layout holder

lblTitle = new Label(" ", Label.CENTER);

lblName = new Label("Student Name:");
lblName.setFont(labelFont);
txtName = new TextField(20);
txtName.setFont(inputFont);

lblRegNo = new Label("Register Number:");
lblRegNo.setFont(labelFont);
txtRegNo = new TextField(20);
txtRegNo.setFont(inputFont);

lblDept = new Label("Department:");
lblDept.setFont(labelFont);
txtDept = new TextField(20);
txtDept.setFont(inputFont);

lblGender = new Label("Gender:");
lblGender.setFont(labelFont);
genderGroup = new CheckboxGroup();
chkMale = new Checkbox("Male", genderGroup, true);
chkMale.setFont(inputFont);
chkFemale = new Checkbox("Female", genderGroup, false);
chkFemale.setFont(inputFont);

lblEmail = new Label("Email ID:");
lblEmail.setFont(labelFont);
txtEmail = new TextField(20);
txtEmail.setFont(inputFont);

// Styled Submit Button
```



```

btnSubmit = new Button("Submit Registration");
btnSubmit.setFont(new Font("Segoe UI", Font.BOLD, 14));
btnSubmit.setBackground(new Color(255, 107, 107)); // Coral Vibrant Pink
btnSubmit.setForeground(Color.WHITE);
btnSubmit.addActionListener(this);

// --- GRID POSITIONING ---
// Top structural gap
gbc.gridx = 0; gbc.gridy = 0; gbc.gridwidth = 3;
gbc.ipady = 35; // Leave room for the custom gradient banner
add(lblTitle, gbc);
gbc.ipady = 0; // Reset padding

// Form Fields rows
gbc.gridwidth = 1;
gbc.gridx = 0; gbc.gridy = 1; add(lblName, gbc);
gbc.gridx = 1; gbc.gridwidth = 2; add(txtName, gbc);

gbc.gridwidth = 1;
gbc.gridx = 0; gbc.gridy = 2; add(lblRegNo, gbc);
gbc.gridx = 1; gbc.gridwidth = 2; add(txtRegNo, gbc);

gbc.gridwidth = 1;
gbc.gridx = 0; gbc.gridy = 3; add(lblDept, gbc);
gbc.gridx = 1; gbc.gridwidth = 2; add(txtDept, gbc);

gbc.gridwidth = 1;
gbc.gridx = 0; gbc.gridy = 4; add(lblGender, gbc);
gbc.gridx = 1; add(chkMale, gbc);
gbc.gridx = 2; add(chkFemale, gbc);

gbc.gridx = 0; gbc.gridy = 5; add(lblEmail, gbc);

```

```
gbc.gridx = 1; gbc.gridwidth = 2; add(txtEmail, gbc);
```

```
// Submit Button Row
```

```
gbc.gridx = 0; gbc.gridy = 6; gbc.gridwidth = 3;
```

```
gbc.insets = new Insets(20, 15, 20, 15);
```

```
add(btnSubmit, gbc);
```

```
}
```

```
@Override
```

```
public void actionPerformed(ActionEvent e) {
```

```
    if (e.getSource() == btnSubmit) {
```

```
        displayName = txtName.getText().trim();
```

```
        displayRegNo = txtRegNo.getText().trim();
```

```
        displayDept = txtDept.getText().trim();
```

```
        displayGender = genderGroup.getSelectedCheckbox().getLabel();
```

```
        displayEmail = txtEmail.getText().trim();
```

```
        isSubmitted = true;
```

```
        repaint(); // Triggers paint method to draw results smoothly
```

```
    }
```

```
}
```

```
@Override
```

```
public void paint(Graphics g) {
```

```
    Graphics2D g2d = (Graphics2D) g;
```

```
    // Turn on Anti-Aliasing for silky smooth text and shapes
```

```
    g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,  
    RenderingHints.VALUE_ANTIALIAS_ON);
```

```
// 1. Draw Base Applet Background
```

```
g2d.setColor(new Color(244, 245, 247)); // Soft minimalist light gray
```

```
g2d.fillRect(0, 0, getWidth(), getHeight());
```

```
// 2. Draw Eye-Catching Header Gradient Banner
```

```
GradientPaint headerGradient = new GradientPaint(0, 0, new Color(78, 84, 200), getWidth(), 65,  
new Color(143, 148, 251));
```

```
g2d.setPaint(headerGradient);
```

```
g2d.fillRect(0, 0, getWidth(), 65);
```

```
// Render Header Title Text
```

```
g2d.setColor(Color.WHITE);
```

```
g2d.setFont(new Font("Segoe UI", Font.BOLD, 18));
```

```
g2d.drawString("UNIVERSITY REGISTRATION", 25, 40);
```

```
// 3. Draw a Beautiful Display Output Card if submitted
```

```
if (isSubmitted) {
```

```
    int cardX = 25;
```

```
    int cardY = 380;
```

```
    int cardWidth = getWidth() - 50;
```

```
    int cardHeight = 170;
```

```
// Soft Shadow effect
```

```
g2d.setColor(new Color(0, 0, 0, 30));
```

```
g2d.fillRoundRect(cardX + 3, cardY + 3, cardWidth, cardHeight, 15, 15);
```

```
// Vibrant Teal/Green Confirmation Header Card Background
```

```
g2d.setColor(Color.WHITE);
```

```
g2d.fillRoundRect(cardX, cardY, cardWidth, cardHeight, 15, 15);
```

```
// Card accent badge headline
```

```
g2d.setColor(new Color(46, 204, 113)); // Emerald Green Accent
```

```
g2d.fillRoundRect(cardX, cardY, cardWidth, 35, 15, 15);
```

```
g2d.fillRect(cardX, cardY + 20, cardWidth, 15); // Flatten inner corners
```

```
g2d.setColor(Color.WHITE);
```

```
g2d.setFont(new Font("Segoe UI", Font.BOLD, 13));
```

```
g2d.drawString("✓ REGISTRATION COMPLETED SUCCESSFULLY", cardX + 15, cardY + 22);
```

```

// Output Grid Details

g2d.setColor(new Color(44, 62, 80)); // Dark Slate text color

g2d.setFont(new Font("Segoe UI", Font.PLAIN, 13));

int textX = cardX + 20;

g2d.drawString("• Name: " + displayName, textX, cardY + 65);
g2d.drawString("• Register No: " + displayRegNo, textX, cardY + 85);
g2d.drawString("• Department: " + displayDept, textX, cardY + 105);
g2d.drawString("• Gender: " + displayGender, textX, cardY + 125);
g2d.drawString("• Email ID: " + displayEmail, textX, cardY + 145);
    }
}
}

```

2. Simple Calculator Applet

Develop an Applet-based calculator that accepts two numbers and provides buttons for Addition, Subtraction, Multiplication, and Division. The application should use TextField, Label, and Button controls. Appropriate event listeners must be implemented so that clicking each button performs the corresponding operation.

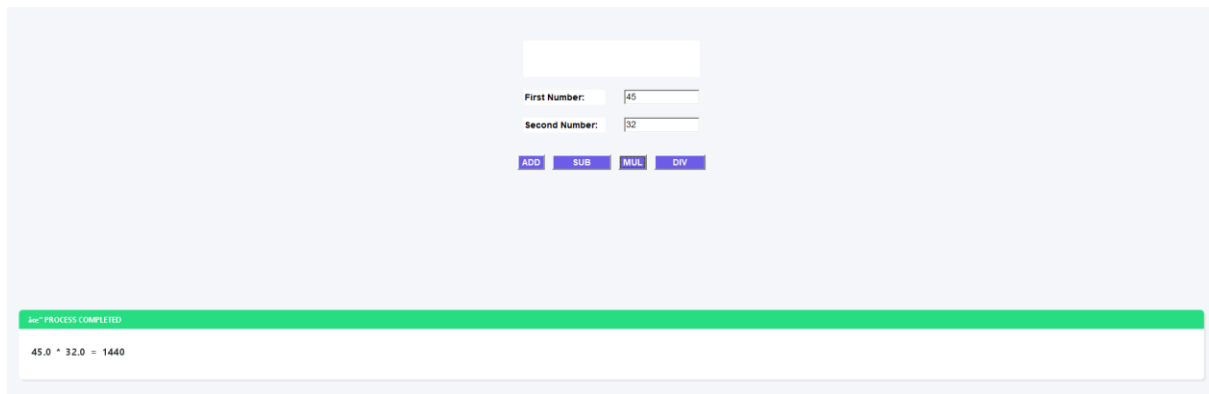
The program must handle invalid numeric input and division by zero appropriately.

The student must demonstrate:

1. Basic Applet structure.
2. AWT controls.
3. Event sources and event objects.
4. ActionListener.
5. Exception handling.
6. Display methods.

MATHEMATICAL CALCULATOR

The screenshot shows a simple graphical user interface for a calculator. At the top, there is a large white rectangular display area. Below this, there are two input fields. The first is labeled "First Number:" and contains the value "10". The second is labeled "Second Number:" and contains the value "34". Below these input fields, there are four buttons arranged horizontally: a blue button with a white plus sign (+), a blue button with a white minus sign (-), a blue button with a white multiplication sign (*), and a blue button with a white division sign (/).



```
import java.applet.Applet;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.ComponentEvent;
import java.awt.event.ComponentListener;

/*
<applet code="SimpleCalculatorApplet.class" width="500" height="450">
</applet>
*/

public class SimpleCalculatorApplet extends Applet implements ActionListener, ComponentListener {

    // Declare AWT Controls
    Label lblTitle, lblNum1, lblNum2;
    TextField txtNum1, txtNum2;
    Button btnAdd, btnSub, btnMul, btnDiv;

    // Display state variables
    String resultMessage = "";
    boolean isError = false;
    boolean hasCalculated = false;
```

@Override

```
public void init() {
```

```
    // Listen for window resize or maximize events to update graphics dynamically
```

```
    addComponentListener(this);
```

```
    // GridBagLayout setup for cleaner positioning
```

```
    setLayout(new GridBagLayout());
```

```
    GridBagConstraints gbc = new GridBagConstraints();
```

```
    gbc.insets = new Insets(10, 15, 10, 15);
```

```
    gbc.fill = GridBagConstraints.HORIZONTAL;
```

```
    // Clean typography
```

```
    Font labelFont = new Font("Segoe UI", Font.BOLD, 14);
```

```
    Font inputFont = new Font("Segoe UI", Font.PLAIN, 14);
```

```
    Font buttonFont = new Font("Segoe UI", Font.BOLD, 14);
```

```
    // Spacing holder for the top header banner
```

```
    lblTitle = new Label(" ", Label.CENTER);
```

```
    lblNum1 = new Label("First Number:");
```

```
    lblNum1.setFont(labelFont);
```

```
    txtNum1 = new TextField(12);
```

```
    txtNum1.setFont(inputFont);
```

```
    lblNum2 = new Label("Second Number:");
```

```
    lblNum2.setFont(labelFont);
```

```
    txtNum2 = new TextField(12);
```

```
    txtNum2.setFont(inputFont);
```

```
    // Safe alphanumeric button texts to completely avoid symbol corruption
```

```
    btnAdd = new Button("ADD"); btnAdd.setFont(buttonFont);
```

```
    btnSub = new Button("SUB"); btnSub.setFont(buttonFont);
```

```

btnMul = new Button("MUL"); btnMul.setFont(buttonFont);
btnDiv = new Button("DIV"); btnDiv.setFont(buttonFont);

// Clean deep purple layout theme colors
Color themeColor = new Color(108, 92, 231);
btnAdd.setBackground(themeColor); btnAdd.setForeground(Color.WHITE);
btnSub.setBackground(themeColor); btnSub.setForeground(Color.WHITE);
btnMul.setBackground(themeColor); btnMul.setForeground(Color.WHITE);
btnDiv.setBackground(themeColor); btnDiv.setForeground(Color.WHITE);

// Attach action listeners
btnAdd.addActionListener(this);
btnSub.addActionListener(this);
btnMul.addActionListener(this);
btnDiv.addActionListener(this);

// Arrange elements securely inside grid
// Row 0: Header Space
gbc.gridx = 0; gbc.gridy = 0; gbc.gridwidth = 4;
gbc.ipady = 35; add(lblTitle, gbc); gbc.ipady = 0;

// Row 1: Number 1 Inputs
gbc.gridwidth = 2;
gbc.gridx = 0; gbc.gridy = 1; add(lblNum1, gbc);
gbc.gridx = 2; add(txtNum1, gbc);

// Row 2: Number 2 Inputs
gbc.gridx = 0; gbc.gridy = 2; add(lblNum2, gbc);
gbc.gridx = 2; add(txtNum2, gbc);

// Row 3: Action Buttons Panel Row
gbc.gridwidth = 1; gbc.insets = new Insets(25, 6, 15, 6);

```

```
gbc.gridx = 0; gbc.gridy = 3; add(btnAdd, gbc);  
gbc.gridx = 1; add(btnSub, gbc);  
gbc.gridx = 2; add(btnMul, gbc);  
gbc.gridx = 3; add(btnDiv, gbc);  
}
```

@Override

```
public void actionPerformed(ActionEvent e) {  
    hasCalculated = true;  
    isError = false;  
  
    try {  
        double n1 = Double.parseDouble(txtNum1.getText().trim());  
        double n2 = Double.parseDouble(txtNum2.getText().trim());  
        double output = 0;  
        Object source = e.getSource();  
  
        if (source == btnAdd) {  
            output = n1 + n2;  
            resultMessage = n1 + " + " + n2 + " = " + formatResult(output);  
        } else if (source == btnSub) {  
            output = n1 - n2;  
            resultMessage = n1 + " - " + n2 + " = " + formatResult(output);  
        } else if (source == btnMul) {  
            output = n1 * n2;  
            resultMessage = n1 + " * " + n2 + " = " + formatResult(output);  
        } else if (source == btnDiv) {  
            if (n2 == 0) {  
                throw new ArithmeticException();  
            }  
            output = n1 / n2;  
            resultMessage = n1 + " / " + n2 + " = " + formatResult(output);  
        }  
    }  
}
```



```

    }

    } catch (NumberFormatException ex) {
        isError = true;
        resultMessage = "ERROR: Please enter valid digits!";
    } catch (ArithmeticException ex) {
        isError = true;
        resultMessage = "ERROR: Cannot divide by zero!";
    }

    repaint();
}

// Drops any ugly trailing formatting decimals smoothly
private String formatResult(double d) {
    if (d == (long) d) return String.format("%d", (long) d);
    return String.format("%.4f", d).replaceAll("0+$", "").replaceAll("\\.$", "");
}

@Override
public void paint(Graphics g) {
    Graphics2D g2d = (Graphics2D) g;

    g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
        RenderingHints.VALUE_ANTIALIAS_ON);

    int currentWidth = getWidth();
    int currentHeight = getHeight();

    // 1. Solid modern gray panel backdrop
    g2d.setColor(new Color(245, 246, 250));
    g2d.fillRect(0, 0, currentWidth, currentHeight);

    // 2. Responsive header banner expanding dynamically with window width

```

```

    GradientPaint bannerGrad = new GradientPaint(0, 0, new Color(116, 185, 255), currentWidth,
65, new Color(108, 92, 231));

    g2d.setPaint(bannerGrad);
    g2d.fillRect(0, 0, currentWidth, 65);

    // Header Title
    g2d.setColor(Color.WHITE);
    g2d.setFont(new Font("Segoe UI", Font.BOLD, 18));
    g2d.drawString("MATHEMATICAL CALCULATOR", 25, 40);

    // 3. Dynamic layout calculation for the output display card
    if (hasCalculated) {
        int cardX = 20;
        // Place card dynamically near the bottom relative to window height
        int cardY = Math.max(280, currentHeight - 150);
        int cardW = currentWidth - 40;
        int cardH = 110;

        // Soft shadow layer
        g2d.setColor(new Color(0, 0, 0, 15));
        g2d.fillRoundRect(cardX + 3, cardY + 3, cardW, cardH, 12, 12);

        // Crisp white inner card frame
        g2d.setColor(Color.WHITE);
        g2d.fillRoundRect(cardX, cardY, cardW, cardH, 12, 12);

        // Responsive state accent header strip
        Color statusColor = isError ? new Color(235, 77, 75) : new Color(38, 222, 129);
        g2d.setColor(statusColor);
        g2d.fillRoundRect(cardX, cardY, cardW, 30, 12, 12);
        g2d.fillRect(cardX, cardY + 15, cardW, 15); // flattens down inner corners cleanly

        // Status string text

```

```
g2d.setColor(Color.WHITE);

g2d.setFont(new Font("Segoe UI", Font.BOLD, 12));

g2d.drawString(isError ? "⚠ SYSTEM EXCEPTION ENCOUNTERED" : "✓ PROCESS COMPLETED",
cardX + 15, cardY + 20);
```

```
// Calculated mathematical formula text

g2d.setColor(isError ? new Color(214, 48, 49) : new Color(45, 52, 54));

g2d.setFont(new Font("Segoe UI", Font.BOLD, 15));

g2d.drawString(resultMessage, cardX + 20, cardY + 72);

}

}
```

```
// --- ComponentListener Interface Implementation for Auto-Repaint on Resize/Maximize ---

@Override public void componentResized(ComponentEvent e) { repaint(); }

@Override public void componentMoved(ComponentEvent e) {}

@Override public void componentShown(ComponentEvent e) {}

@Override public void componentHidden(ComponentEvent e) {}

}
```

3. Course Selection Applet

Develop a university course selection Applet where students can select:

- Department
- Semester
- Course
- Elective type

Use Choice, List, Checkbox, and Button controls wherever appropriate.

When the student clicks Register, display the selected courses and other information.

Use appropriate layout managers and event listeners.

Applet Viewer: CourseSelectionApplet.class

Applet

CAMPUS COURSE SELECTION

Select Department: Computer Science & Engineering

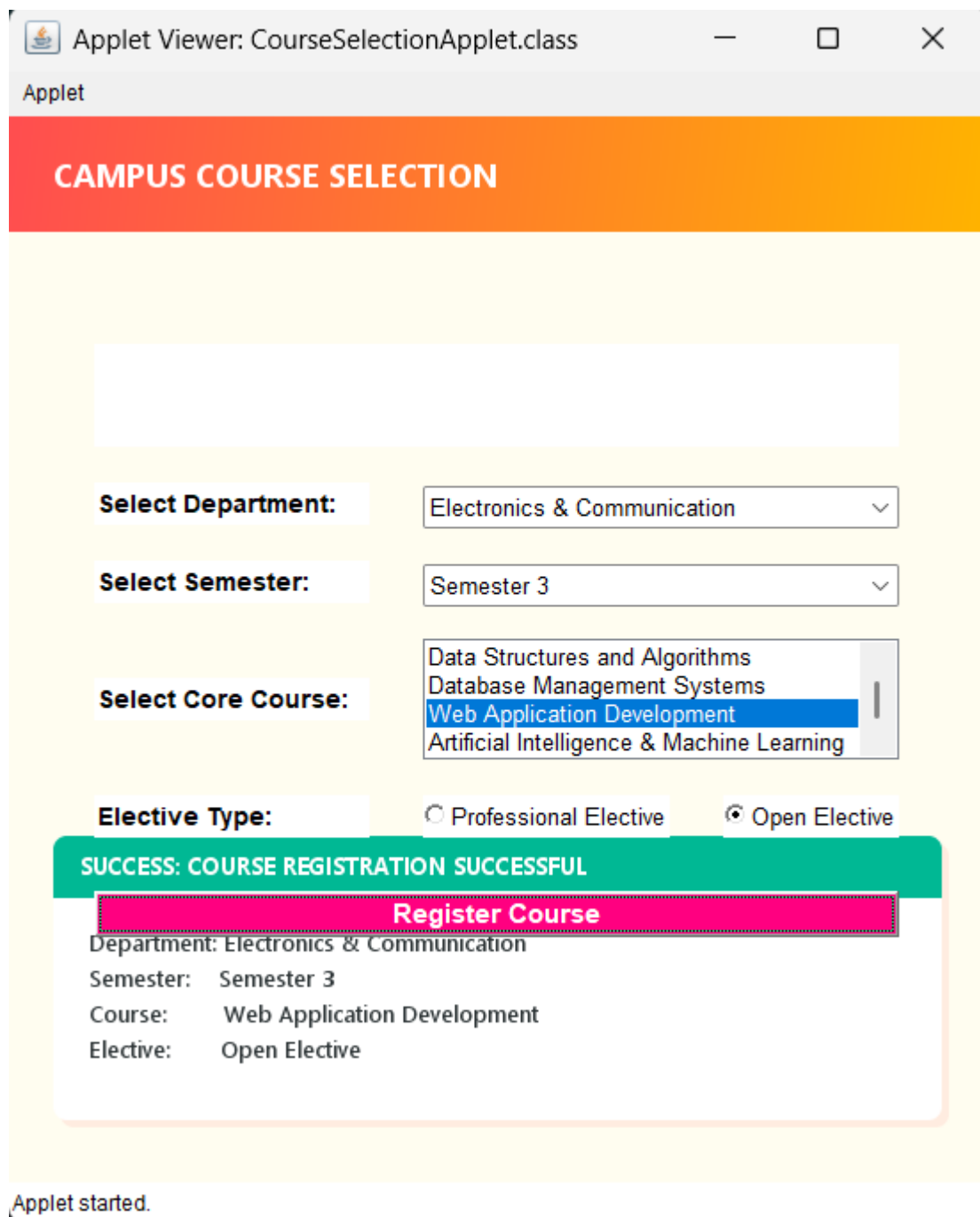
Select Semester: Semester 1

Select Core Course: Object Oriented Programming with Java
Data Structures and Algorithms
Database Management Systems
Web Application Development

Elective Type: ☒ Professional Elective ☐ Open Elective

Register Course

Applet started.



```
import java.applet.Applet;  
import java.awt.*;  
import java.awt.event.ActionEvent;  
import java.awt.event.ActionListener;  
import java.awt.event.ComponentEvent;  
import java.awt.event.ComponentListener;
```

```
/*
```

```
<applet code="CourseSelectionApplet.class" width="550" height="600">
```

```
</applet>
```

```
*/
```

```
public class CourseSelectionApplet extends Applet implements ActionListener, ComponentListener {
```

```
    // Declare required AWT Controls
```

```
    Label lblTitle, lblDept, lblSem, lblCourse, lblElective;
```

```
    Choice choiceDept, choiceSem;
```

```
    List listCourses;
```

```
    CheckboxGroup electiveGroup;
```

```
    Checkbox chkProfessional, chkOpen;
```

```
    Button btnRegister;
```

```
    // Display storage variables
```

```
    String selectedDept = "";
```

```
    String selectedSem = "";
```

```
    String selectedCourse = "";
```

```
    String selectedElective = "";
```

```
    boolean isRegistered = false;
```

```
    @Override
```

```
    public void init() {
```

```
        // Handle screen resizing and maximizing smoothly
```

```
        addComponentListener(this);
```

```
        // GridBagLayout setup for cleaner positioning
```

```
        setLayout(new GridBagLayout());
```

```
        GridBagConstraints gbc = new GridBagConstraints();
```

```
        gbc.insets = new Insets(10, 15, 10, 15);
```

```
        gbc.fill = GridBagConstraints.HORIZONTAL;
```

```
// Custom Fonts

Font labelFont = new Font("Segoe UI", Font.BOLD, 14);
Font inputFont = new Font("Segoe UI", Font.PLAIN, 13);
Font buttonFont = new Font("Segoe UI", Font.BOLD, 15);


// Blank spacer for the custom header banner
lblTitle = new Label(" ", Label.CENTER);


// 1. Department Selection using a Choice Dropdown
lblDept = new Label("Select Department:");
lblDept.setFont(labelFont);
choiceDept = new Choice();
choiceDept.setFont(inputFont);
choiceDept.add("Computer Science & Engineering");
choiceDept.add("Information Technology");
choiceDept.add("Electronics & Communication");
choiceDept.add("Mechanical Engineering");


// 2. Semester Selection using a Choice Dropdown
lblSem = new Label("Select Semester:");
lblSem.setFont(labelFont);
choiceSem = new Choice();
choiceSem.setFont(inputFont);
for (int i = 1; i <= 8; i++) {
    choiceSem.add("Semester " + i);
}


// 3. Course Selection using a List box component
lblCourse = new Label("Select Core Course:");
lblCourse.setFont(labelFont);
listCourses = new List(4, false); // Shows 4 rows, false means single choice
listCourses.setFont(inputFont);
```

```

listCourses.add("Object Oriented Programming with Java");
listCourses.add("Data Structures and Algorithms");
listCourses.add("Database Management Systems");
listCourses.add("Web Application Development");
listCourses.add("Artificial Intelligence & Machine Learning");
listCourses.select(0); // Select the first item by default

// 4. Elective Type Selection using Checkboxes
lblElective = new Label("Elective Type:");
lblElective.setFont(labelFont);
electiveGroup = new CheckboxGroup();
chkProfessional = new Checkbox("Professional Elective", electiveGroup, true);
chkProfessional.setFont(inputFont);
chkOpen = new Checkbox("Open Elective", electiveGroup, false);
chkOpen.setFont(inputFont);

// 5. Bright Colorful Registration Button
btnRegister = new Button("Register Course");
btnRegister.setFont(buttonFont);
btnRegister.setBackground(new Color(255, 0, 128)); // Bright Eye-Catching Pink
btnRegister.setForeground(Color.WHITE);
btnRegister.addActionListener(this);

// --- GRID POSITIONING LAYER ---
// Row 0: Banner Space
gbc.gridx = 0; gbc.gridy = 0; gbc.gridwidth = 3;
gbc.ipady = 35; add(lblTitle, gbc); gbc.ipady = 0;

// Row 1: Department
gbc.gridwidth = 1;
gbc.gridx = 0; gbc.gridy = 1; add(lblDept, gbc);
gbc.gridx = 1; gbc.gridwidth = 2; add(choiceDept, gbc);

```



```

// Row 2: Semester
gbc.gridwidth = 1;
gbc.gridx = 0; gbc.gridy = 2; add(lblSem, gbc);
gbc.gridx = 1; gbc.gridwidth = 2; add(choiceSem, gbc);

// Row 3: Course List
gbc.gridwidth = 1;
gbc.gridx = 0; gbc.gridy = 3; add(lblCourse, gbc);
gbc.gridx = 1; gbc.gridwidth = 2; add(listCourses, gbc);

// Row 4: Elective Radio Checkboxes
gbc.gridwidth = 1;
gbc.gridx = 0; gbc.gridy = 4; add(lblElective, gbc);
gbc.gridx = 1; add(chkProfessional, gbc);
gbc.gridx = 2; add(chkOpen, gbc);

// Row 5: Register Action Button
gbc.gridx = 0; gbc.gridy = 5; gbc.gridwidth = 3;
gbc.insets = new Insets(20, 15, 20, 15);
add(btnRegister, gbc);
}

@Override
public void actionPerformed(ActionEvent e) {
    if (e.getSource() == btnRegister) {
        // Read values safely from custom selection fields
        selectedDept = choiceDept.getSelectedItemAt();
        selectedSem = choiceSem.getSelectedItemAt();
        selectedCourse = listCourses.getSelectedItemAt();

        // Fallback check if nothing was actively selected in the list
    }
}

```

```

        if (selectedCourse == null) {
            selectedCourse = "None Selected";
        }

        selectedElective = electiveGroup.getSelectedCheckbox().getLabel();
        isRegistered = true;

        // Force redraw to render the confirmation screen details
        repaint();
    }
}

@Override
public void paint(Graphics g) {
    Graphics2D g2d = (Graphics2D) g;

    g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
        RenderingHints.VALUE_ANTIALIAS_ON);

    int currentWidth = getWidth();
    int currentHeight = getHeight();

    // 1. Bright cheerful layout background color
    g2d.setColor(new Color(255, 253, 240)); // Bright pastel warm yellow hue
    g2d.fillRect(0, 0, currentWidth, currentHeight);

    // 2. High-energy colorful gradient heading banner
    GradientPaint topBannerGrad = new GradientPaint(0, 0, new Color(255, 78, 80), currentWidth,
        65, new Color(255, 180, 0));
    g2d.setPaint(topBannerGrad);
    g2d.fillRect(0, 0, currentWidth, 65);

    // Render clean layout title text
    g2d.setColor(Color.WHITE);

```

```

g2d.setFont(new Font("Segoe UI", Font.BOLD, 18));
g2d.drawString("CAMPUS COURSE SELECTION", 25, 40);

// 3. Render Output Registration Details Display Panel Card
if (isRegistered) {
    int cardX = 25;

    int cardY = Math.max(390, currentHeight - 195); // Dynamic layout positioning for maximizing
    int cardW = currentWidth - 50;
    int cardH = 160;

    // Vibrant Neon drop-shadow container layer
    g2d.setColor(new Color(255, 78, 80, 25));
    g2d.fillRoundRect(cardX + 4, cardY + 4, cardW, cardH, 15, 15);

    // Clean crisp white inner panel background frame
    g2d.setColor(Color.WHITE);
    g2d.fillRoundRect(cardX, cardY, cardW, cardH, 15, 15);

    // Colorful Cyan-Teal Confirmation Status Headline Ribbon
    g2d.setColor(new Color(0, 184, 148));
    g2d.fillRoundRect(cardX, cardY, cardW, 35, 15, 15);
    g2d.fillRect(cardX, cardY + 20, cardW, 15); // Flattens inner corners perfectly

    g2d.setColor(Color.WHITE);
    g2d.setFont(new Font("Segoe UI", Font.BOLD, 13));
    g2d.drawString("SUCCESS: COURSE REGISTRATION SUCCESSFUL", cardX + 15, cardY + 22);

    // Text selections display layout grid block
    g2d.setColor(new Color(45, 52, 54)); // High-contrast charcoal text
    g2d.setFont(new Font("Segoe UI", Font.PLAIN, 13));

    int textX = cardX + 20;

```

```

        g2d.drawString("Department: " + selectedDept, textX, cardY + 65);
        g2d.drawString("Semester:  " + selectedSem, textX, cardY + 85);
        g2d.drawString("Course:     " + selectedCourse, textX, cardY + 105);
        g2d.drawString("Elective:   " + selectedElective, textX, cardY + 125);
    }
}

// --- Repaint loop adjustments for sizing changes or full screen maximizing ---
@Override public void componentResized(ComponentEvent e) { repaint(); }
@Override public void componentMoved(ComponentEvent e) {}
@Override public void componentShown(ComponentEvent e) {}
@Override public void componentHidden(ComponentEvent e) {}
}

```

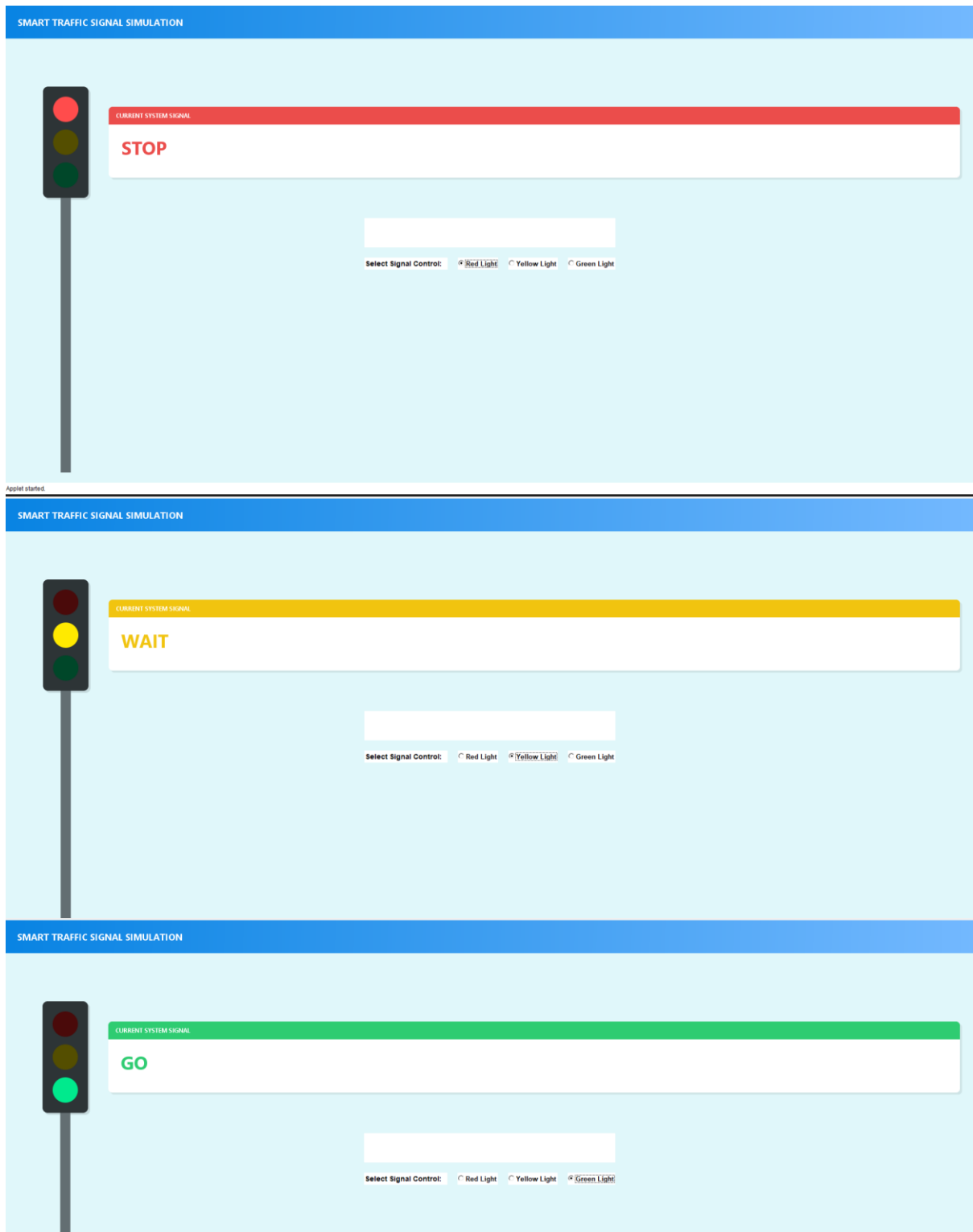
4. Traffic Signal Control Applet

Develop an AWT Applet that simulates a traffic signal with three options:

- Red
- Yellow
- Green

Use appropriate controls to allow the user to select a signal. When the user selects a signal, display the corresponding message such as STOP, WAIT, or GO.

Use appropriate event sources, event classes, and listener interfaces.



```
import java.applet.Applet;  
import java.awt.*;  
import java.awt.event.ItemEvent;  
import java.awt.event.ItemListener;  
import java.awt.event.ComponentEvent;
```

```

import java.awt.event.ComponentListener;

/*
<applet code="TrafficSignalApplet.class" width="550" height="550">
</applet>
*/

public class TrafficSignalApplet extends Applet implements ItemListener, ComponentListener {

    // Declare AWT Selection Controls
    Label lblTitle, lblSelect;
    CheckboxGroup signalGroup;
    Checkbox chkRed, chkYellow, chkGreen;

    // State tracking variables
    String activeSignal = "RED"; // Default starting light
    String activeMessage = "STOP";
    Color messageColor = new Color(235, 77, 75);

    @Override
    public void init() {
        // Attach component listener to handle screen maximization resizing
        addComponentListener(this);

        // GridBagLayout configuration
        setLayout(new GridBagLayout());
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(10, 10, 10, 10);
        gbc.fill = GridBagConstraints.HORIZONTAL;

        Font labelFont = new Font("Segoe UI", Font.BOLD, 14);
        Font radioFont = new Font("Segoe UI", Font.BOLD, 13);
    }

```

```
// Blank spacing placeholder for top header banner space
lblTitle = new Label(" ", Label.CENTER);

lblSelect = new Label("Select Signal Control:");
lblSelect.setFont(labelFont);

// Create Checkbox Group for exclusive radio selection options
signalGroup = new CheckboxGroup();
chkRed = new Checkbox("Red Light", signalGroup, true);
chkRed.setFont(radioFont);
chkYellow = new Checkbox("Yellow Light", signalGroup, false);
chkYellow.setFont(radioFont);
chkGreen = new Checkbox("Green Light", signalGroup, false);
chkGreen.setFont(radioFont);

// Attach appropriate item selection listeners
chkRed.addItemListener(this);
chkYellow.addItemListener(this);
chkGreen.addItemListener(this);

// --- POSITIONING GRID ELEMENTS ---
// Row 0: Banner Area
gbc.gridx = 0; gbc.gridy = 0; gbc.gridwidth = 4;
gbc.ipady = 35; add(lblTitle, gbc); gbc.ipady = 0;

// Row 1: Selection Interface Row
gbc.gridwidth = 1;
gbc.gridx = 0; gbc.gridy = 1; add(lblSelect, gbc);
gbc.gridx = 1; add(chkRed, gbc);
gbc.gridx = 2; add(chkYellow, gbc);
gbc.gridx = 3; add(chkGreen, gbc);
```

```
}
```

```
// Handle Item Selection Events
```

```
@Override
```

```
public void itemStateChanged(ItemEvent e) {
```

```
    Checkbox selected = signalGroup.getSelectedCheckbox();
```

```
    if (selected == chkRed) {
```

```
        activeSignal = "RED";
```

```
        activeMessage = "STOP";
```

```
        messageColor = new Color(235, 77, 75); // Radiant Red
```

```
    } else if (selected == chkYellow) {
```

```
        activeSignal = "YELLOW";
```

```
        activeMessage = "WAIT";
```

```
        messageColor = new Color(241, 196, 15); // Vibrant Yellow
```

```
    } else if (selected == chkGreen) {
```

```
        activeSignal = "GREEN";
```

```
        activeMessage = "GO";
```

```
        messageColor = new Color(46, 204, 113); // Neon Green
```

```
    }
```

```
// Force a graphic re-paint loop update
```

```
repaint();
```

```
}
```

```
@Override
```

```
public void paint(Graphics g) {
```

```
    Graphics2D g2d = (Graphics2D) g;
```

```
    g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,  
RenderingHints.VALUE_ANTIALIAS_ON);
```

```
int W = getWidth();
```

```
int H = getHeight();
```



```

// 1. Fill a bright, colorful sky-blue canvas backdrop
g2d.setColor(new Color(224, 247, 250));
g2d.fillRect(0, 0, W, H);

// 2. High-energy colorful gradient heading banner
GradientPaint topBanner = new GradientPaint(0, 0, new Color(9, 132, 227), W, 65, new
Color(116, 185, 255));
g2d.setPaint(topBanner);
g2d.fillRect(0, 0, W, 65);

// Render header string text
g2d.setColor(Color.WHITE);
g2d.setFont(new Font("Segoe UI", Font.BOLD, 18));
g2d.drawString("SMART TRAFFIC SIGNAL SIMULATION", 25, 40);

// 3. Draw the Central Traffic Light Housing structure
int poleX = 75;
int poleY = 160;
int boxW = 90;
int boxH = 220;

// Draw shadow under light pole box housing
g2d.setColor(new Color(0, 0, 0, 30));
g2d.fillRoundRect(poleX + 4, poleY + 4, boxW, boxH, 20, 20);

// Main dark metallic housing panel body box
g2d.setColor(new Color(45, 52, 54));
g2d.fillRoundRect(poleX, poleY, boxW, boxH, 20, 20);

// Lower support pole fixture structure
g2d.setColor(new Color(99, 110, 114));
g2d.fillRect(poleX + 35, poleY + boxH, 20, H - (poleY + boxH) - 20);

```

```

// Define exact color variables based on system state logic definitions
Color redLamp = activeSignal.equals("RED") ? new Color(255, 76, 76) : new Color(74, 10, 10);
Color yellowLamp = activeSignal.equals("YELLOW") ? new Color(255, 234, 0) : new Color(84, 78,
0);
Color greenLamp = activeSignal.equals("GREEN") ? new Color(0, 234, 141) : new Color(0, 71, 41);

// Render 3 illuminated signal circular lens lamps
int radius = 50;
int startX = poleX + 20;

g2d.setColor(redLamp);
g2d.fillOval(startX, poleY + 20, radius, radius);

g2d.setColor(yellowLamp);
g2d.fillOval(startX, poleY + 85, radius, radius);

g2d.setColor(greenLamp);
g2d.fillOval(startX, poleY + 150, radius, radius);

// 4. Render Right-Side Command Information Alert Status Card
int cardX = poleX + boxW + 40;
int cardY = 200;
int cardW = Math.max(200, W - cardX - 30);
int cardH = 140;

// Display card outer shadow plate
g2d.setColor(new Color(0, 0, 0, 20));
g2d.fillRoundRect(cardX + 4, cardY + 4, cardW, cardH, 15, 15);

// White base container card plate
g2d.setColor(Color.WHITE);
g2d.fillRoundRect(cardX, cardY, cardW, cardH, 15, 15);

```

```

// Draw State Indicator top status header band line inside card frame
g2d.setColor(messageColor);
g2d.fillRoundRect(cardX, cardY, cardW, 35, 15, 15);
g2d.fillRect(cardX, cardY + 20, cardW, 15); // flattens inside down lower box corners

g2d.setColor(Color.WHITE);
g2d.setFont(new Font("Segoe UI", Font.BOLD, 12));
g2d.drawString("CURRENT SYSTEM SIGNAL", cardX + 15, cardY + 22);

// Main command instruction large status text word print loop
g2d.setColor(messageColor);
g2d.setFont(new Font("Segoe UI", Font.BOLD, 36));
g2d.drawString(activeMessage, cardX + 25, cardY + 95);
}

// --- ComponentListener Methods to handle full window repaint layout refreshes ---
@Override public void componentResized(ComponentEvent e) { repaint(); }
@Override public void componentMoved(ComponentEvent e) {}
@Override public void componentShown(ComponentEvent e) {}
@Override public void componentHidden(ComponentEvent e) {}
}

```